

Tool Experience Report: AutoGen and MetaGPT

Oliver Browne

Papers: Wu et al., *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*; Hong et al., *MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework* (ICLR '24)

Tools used: AutoGen AgentChat 0.7.5 and MetaGPT 0.8.2

Date of experiments: 30 September 2026

An AI coding assistant ran the commands on my Windows machine. I kept every prompt, response, generated file and test log. Nothing was mocked, and I didn't edit any generated code before testing it.

1 Your experience using the tool

Setup

I had no paid API key, so I used Qwen2.5-Coder-1.5B (4-bit) through llama.cpp in place of GPT-4. It ran on the CPU of a Ryzen 7 5800X with 32 GB of RAM. This can't reproduce the papers' numbers, but it shows how much depends on the framework and how much on the model. Both tools got the same task, `merge_intervals`, which has four traps: don't change the input, don't share its inner lists, don't shrink a containing interval, and reject reversed intervals. I wrote the tests before any model call.

AutoGen installed in about six seconds. Version 0.7 is a rewrite, though, so the paper's code no longer runs. Using a local model only needed a `base_url` and a dummy API key. The code executor failed inside my Windows sandbox but worked outside it.

MetaGPT took eight tries to install, and six failed. One pinned package no longer exists on PyPI, so pip quietly fell back to MetaGPT 0.1. Another package failed to build because of Windows path limits. The working install needed Python 3.11, two manual overrides and 218 packages.

AutoGen

I set up a coder, a reviewer and a test runner with no LLM, taking turns in a group chat. The coder fixed one bug but added a new one, and it stayed at 8 of 11 tests. After seeing the failures, it blamed a test that was passing and resent the same code. The reviewer approved every version and claimed the code didn't change the input, which was false. The test runner was the only participant that reported correctly.

MetaGPT

I built a two-role team from the official tutorial, with one role writing code and the other reviewing it. This tests MetaGPT’s message routing but not the document pipeline the paper is about. The run took 4.65 seconds and two LLM calls. The code changed the input and tried to edit tuples, so it passed 3 of 10 tests. The reviewer’s whole answer was **PASS**.

2 Strengths

AutoGen lets a plain program, such as a test runner, join the chat as an agent, which is how I got honest test results into the loop. AutoGen was also easy to set up, and its logs showed what each agent did and why the run stopped. MetaGPT’s roles react to message types, so a workflow takes a line or two to describe. It makes one call per role, which let it finish in 5 seconds against AutoGen’s 77.

3 Weaknesses

In both tools the LLM reviewer approved broken code, and neither framework runs tests by default. AutoGen has changed so much that the paper no longer matches the code. It also can’t tell when an agent keeps repeating the same answer. MetaGPT’s packaging is fragile, and the silent fallback to an old version could fool a new user. Its full pipeline expects a large model and has no setting for small ones.

4 Potential for practical impact

Both tools turn LLM workflows into code you can version and test. AutoGen suits teams that want people or tools in the loop. MetaGPT suits quick prototypes, and it’s useful for teaching what design documents look like. Trust is the biggest barrier, since a reviewer that approves broken code gives false confidence. Cost, broken installs and the safety of running generated code also slow adoption.

5 Reflection and recommendations

Both frameworks routed messages and ended runs correctly, and every quality failure came from the agents. In AutoGen the reviewer never corrected anything and only added false approvals. Running the tools side by side showed the trade-off: AutoGen can retry but is slow, while MetaGPT is fast and can’t recover. AutoGen should detect when an agent repeats the same code and stop or ask a human. MetaGPT should run tests by default and move its cloud packages into optional extras.

Evidence summary

	AutoGen 0.7.5	MetaGPT 0.8.2
Install attempts / failures	2 / 1 (network only)	8 / 6
Packages installed	28	218 (+2 overrides)
Python	3.13	3.11 (requires < 3.12)
Roles	coder, reviewer, non-LLM verifier	Implementer, Reviewer
LLM calls / wall time	6 / 76.8s	2 / 4.65s (team)
Final tests passing	8 / 11 (unchanged from baseline)	3 / 10
Reviewer verdict	“correct” ×3 (wrong)	PASS (wrong)
Task outcome	Fail	Fail